

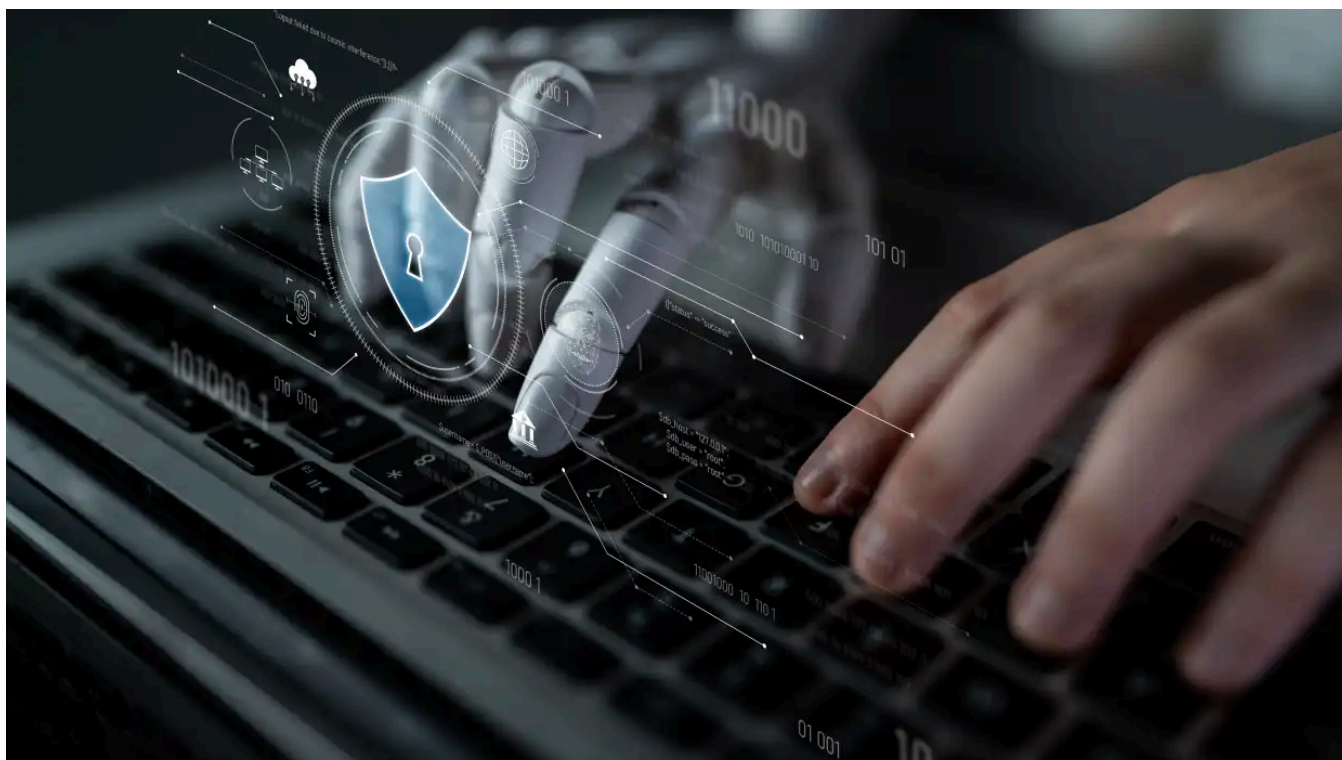
Автор: Тарин Пламб

Будущее исследований в области безопасности ИИ — не за автономностью, а за усилением человеческого фактора

Новости

10 августа 2026 года • 6 минут

HTTP Terminator сгенерировал, оценил и применил на практике тысячи гипотез о десинхронизации HTTP, а также выявил новую технику, которую ни искусственный интеллект, ни его оператор не смогли бы найти самостоятельно.



Источник: Summit Art Creations / Shutterstock

Познакомьтесь с HTTP Terminator — новой системой искусственного интеллекта, которая выявила сотни веб-сайтов, уязвимых для подмены HTTP-запросов, взломала их в режиме реального времени и даже обнаружила «совершенно новый класс» уязвимостей, получивший название «confusion of shared-parser».

Но он сделал это не сам: все это время им управлял человек, и это, пожалуй, самое интересное открытие из всех.

Исследователь из компании PortSwigger, специализирующейся на кибербезопасности, разработал собственный процесс проектирования и создания ИИ под названием HTTP Terminator. Он задавал узкие, но важные вопросы, исключал слабые ответы, применял логику обнаружения аномалий, использовал детерминированный код для ограничения поведения агента и применял полученные результаты в последующих «каскадных» исследованиях.

«Это переворачивает общепринятое представление с ног на голову, показывая, что эксперт может стать мощным катализатором для исследовательской системы на основе ИИ, — объяснил Джеймс Кеттл [<https://portswigger.net/research/james-kettle>], директор по исследованиям PortSwigger, в официальном документе [<https://portswigger.net/research/can-ai-do-novel-security-research>]. — Человек, вовлеченный в процесс, все еще может принести значительную пользу, в отличие от тех, кто просто создает систему, а потом отходит в сторону».

Нарушение потока HTTP-запросов

Атаки с десинхронизацией HTTP, также известные как подмена HTTP-запросов [<https://portswigger.net/web-security/request-smuggling>], нарушают процесс обработки веб-сайтами HTTP-запросов от различных пользователей. В этой архитектуре пользователи отправляют запросы на внешний сервер, который затем перенаправляет их на внутренние серверы.

Проблема заключается в изначально «слабой» изоляции запросов, отметил Кеттл. Фронтенд-сервер обычно отправляет несколько запросов одновременно по одному и тому же серверному сетевому соединению для повышения производительности и эффективности.

«HTTP-запросы отправляются один за другим, и принимающий сервер должен определить, где заканчивается один запрос и начинается следующий», — объяснил он. Но фронтенд и бэкенд должны «договариваться» о том, где заканчивается каждый запрос, иначе злоумышленники могут отправлять неоднозначные сообщения, которые бэкенд интерпретирует как два запроса.

С помощью подмены ответных запросов (Response Query Poisoning, RQP) злоумышленники могут внедрять ложную информацию в последующие запросы или иным образом манипулировать потоками данных, чтобы системы теряли представление о том, откуда пришли те или иные ответы. Затем они могут перехватывать ответы, предназначенные для других пользователей, которые иногда содержат конфиденциальную информацию, учетные данные или ключи API.

Хотя подмена запросов обычно связана с запросами по протоколу HTTP/1, веб-сайты, поддерживающие HTTP/2, также могут быть уязвимы в зависимости от архитектуры их серверной части, отметил Кеттл.

«Уязвимости, связанные с подменой запросов, часто носят критический характер и позволяют злоумышленнику обойти средства защиты, получить несанкционированный доступ к конфиденциальным данным и напрямую скомпрометировать других пользователей приложения», — сказал он.

Методология HTTP Terminator

Кеттл создал HTTP Terminator на основе собственных исследовательских методологий.

Первый этап — генерация идей: система самостоятельно генерирует гипотетические тестируемые RQP-атаки, такие как, например, триггеры десинхронизации, паттерны или методы использования в качестве оружия.

The next step is evaluation [<https://www.csoononline.com/article/4206598/human-oversight-is-still-critical-as-ai-patching-tools-miss-security-risks.html>]: hypotheses are tested at large scale to identify which actually worked. HTTP Terminator used live websites that allow security

testing via bug-bounty and Vulnerability Disclosure Programs (VDPs). A built-in anomaly detection layer flagged unusual responses.

Following that is a weaponization phase, which determines whether findings actually have real-life impact.

In some cases, they did: HTTP Terminator initially generated 30,000 unique **attack vectors** [<https://www.csoonline.com/article/4204731/attackers-are-crafting-malicious-ai-instruction-files-to-turn-your-agentic-workflows-into-quiet-criminal-helpers.html>] that it tested continuously, soon identifying 700 vulnerable targets, including financial institutions and government infrastructure. The system even stole a live API key from one of these banks, and other breaches were traced back to flaws in popular enterprise products like Apache Traffic Server, Citrix NetScaler, and BeyondTrust.

Finally, there's a step Kettle calls 'cascade.' Each finding may be a clue to an overlooked target or detection path. "When you discover something, if you explore back up the tree you may find other undiscovered branches," he explained.

For instance: How could you detect similar behavior elsewhere? And could the behavior enable other types of attacks? Essentially, each finding becomes "the seed for the next."

"That might not look like much, but it creates a positive feedback loop which can spiral into a cascade of discoveries taking you beyond predictable findings, into the unknown," Kettle said.

Ultimately, "I realized that autonomous vs human is the wrong framing," he explained. "It's better to frame system design as AI vs code vs human."

A new tool for security teams

HTTP Terminator was able to invent and prove several novel desync triggers, one novel desync pattern, and a desync weaponization technique.

Notably, though, it identified a new "shared-parser confusion" technique that allows attackers to parse requests and responses and thus increase their attack surface.

"That, by itself, is absolutely huge," Kettle said. But, he emphasized, "this discovery was not fully autonomous — the HTTP Terminator proposed it, and I validated it. Neither of us would have discovered it alone."

Kettle has open-sourced HTTP Terminator along with his research blueprint so other security teams can "turn their own methodology and instincts into an autonomous research weapon."

The researcher advised tackling tasks in four steps: Set an objective, create an evaluation strategy, establish inspiration sources, and explore cascade routes for new discoveries.

“Evaluation is the first concrete step for both design and implementation,” he said, “because any issues there will derail the entire project.”

It’s also important to “aggressively” identify and resolve data quality issues, as those will be difficult to correct later on. Further, ask high-value questions without being too broad; review outputs in initial test runs and use them to rule out low-value hypotheses; and remember that “every extra sentence of prompt risks context-contamination,” Kettle said.

When a system is fully AI-driven and also reliant on AI-built code, it’s near-impossible for it to improve over time, he said. It’s better to start quickly with an AI-heavy approach, then move responsibility to deterministic code for speed and accuracy.

“Fully autonomous research is real,” Kettle acknowledged. However, “humans are a massive power amplifier for AI research systems.”

Artificial Intelligence[https://www.csoonline.com/artificial-intelligence/]

Vulnerabilities[https://www.csoonline.com/vulnerabilities/]

Security[https://www.csoonline.com/security/]

Hacking[https://www.csoonline.com/hacking/]

Cybercrime[https://www.csoonline.com/cybercrime/]

About

⌵

Policies

⌵

More

⌵

Our Network

⌵

